

UNIT-6: Advance Express JS

File Upload

Multer is a node.js middleware for handling **multipart/form-data**, which is primarily used for uploading files. Multer adds a file or files object to the request object. The file or files object contains the data of files uploaded via the form.

Installation

npm install multer

What is Multipart Data?

The ENCTYPE attribute of <form> tag specifies the method of encoding for the form data. It is one of the two ways of encoding the HTML form. It is specifically used when file uploading is required in HTML form. It sends the form data to server in multiple parts because of large size of file.

- In general, when a “form” is submitted, browsers use “application-x-www-form-urlencoded” content-type.
- This type contains only a list of keys and values and therefore are not capable of uploading files.
- Whereas, when you configure your form to use “multipart/form-data” content-type, browsers will create a “multipart” message where each part will contain a field of the form. A multipart message will consist of text input and file input. This way using multipart/form-data you can upload files.

Multer adds a body object and a file or files object to the request object. The body object contains the values of the text fields of the form, the file or files object contains the files uploaded via the form.

Multer will not process any form which is not multipart (*multipart/form-data*). **Must define in form*

****File information:** Each file contains the following information:

Key	Description
fieldname	Field name specified in the form
originalname	Name of the file on the user’s computer
encoding	Encoding type of the file
mimetype	Mime type of the file (Multipurpose Internet Mail Extensions)
size	Size of the file in bytes
destination	The folder to which the file has been saved

Key	Description
filename	The name of the file within the destination
path	The full path to the uploaded file

DiskStorage() method

The disk storage engine gives you full control on storing files to disk.

There are two options available, destination and filename. They are both functions that determine where the file should be stored.

- **Destination:** It is used to determine within which folder the uploaded files should be stored. This can also be given as a string (e.g. '/tmp/uploads/'). If no destination is given, the operating system's default directory for temporary files is used.
- **Filename:** It is used to determine what the file should be named inside the folder. If no filename is given, each file will be given a random name that doesn't include any file extension.

```
var storage = multer.diskStorage({
  destination: "single",
  filename: function (req, file, cb) {
    cb(null, file.originalname)
  }
})
```

The 2 arguments in cb are

- **null** – Don't want to show any error.
- **file.originalname** – Used the same name of the file as they were uploaded. You can use any name of your choice.

Optional objects of Multer(opts)

Multer accepts an options object, the most basic of which is the dest property, which tells multer where to upload the files. In case you omit the options object, the files will be kept in memory and never written to disk.

By default, Multer will rename the files so as to avoid naming conflicts. The renaming function can be customized according to your needs.

The following are the options that can be passed to Multer.

Key	Description
dest or storage	Where to store the files
limits	Limits of the uploaded data

Key	Description
fileFilter	Function to control which files are accepted
preservePath	Keep the full path of files instead of just the base name

If you want more control over your uploads, you'll want to use the storage option instead of dest.

limits

- ✓ Set File Size Limits
- ✓ When setting file size limits, we need to specify the maximum file size that can be uploaded. The limits option of Multer can be used to set the maximum file size in bytes.

In the example, we set the maximum file size to 1MB (1,000,000 bytes). You can change this to any value that fits your requirements.

Here's an example of how to set file size limits:

```
const upload = multer({storage,
  limits: {
    fileSize: 1000*1000 // 1MB (decimal)
  }
});
```

Or

```
const upload = multer({storage,
  limits: {
    fileSize: 1024*1024 // 1MB (binary)
  }
});
```

In this example, we set the maximum file size to 1MB.

The megabyte is one of several multipliers used to represent larger numbers of bytes. For example, a kilobyte (KB) is equal to 1,000 bytes (decimal) or 1,024 bytes (binary). As such, a megabyte is equal to 1,000 KB (decimal) or 1,024 KB (binary).

No of File Selection options

1. **To upload Single file :** Add `.single(fieldname)` next to `multer()` : Accept a single file with the name fieldname. The single file will be stored in `req.file`.
2. **To upload Multiple file :** Add `.array(fieldname[, maxCount])` : Accept an array of files, all with the name fieldname. Optionally error out if more than maxCount files are uploaded. The array of files will be stored in `req.files`.

Steps to Perform multer Task

Step -1 Create an HTML Form

To create the HTML form, you will use the `< form >` element with the following attributes:

1. **action**: This attribute specifies the URL or route where the form data will be sent when the form is submitted
2. **method**: This attribute specifies the HTTP method to be used when submitting the form. For file uploads, you should use the POST method, as it allows larger amounts of data to be sent. So, set the method attribute to "POST".
3. **enctype**: This attribute specifies the content type used to submit the form data. For file uploads, you need to set the enctype attribute to "multipart/form-data". This encoding type is necessary for browsers to properly handle file uploads.
4. Inside the form, you will add an `< input >` element of type "file". This element allows users to select a file from their local machine. Give the input element a **name** attribute so that it can be identified when the form is submitted (Add "multiple" keyword incase want to upload multiple file). And submit button.

Step -2 Set Up Multer Middleware

Multer is a middleware for handling multipart/form-data requests, specifically designed for file uploads in Node.js.

1. First, import the **multer** module using `require('multer')`. This ensures that you have access to the Multer functionality.
2. Next, define the storage configuration for uploaded files using `multer.diskStorage()`. This configuration determines where the uploaded files will be stored on the server. It takes an object with two functions: **destination** and **filename**.
3. The destination function specifies the directory where the uploaded files will be saved.
4. The filename function determines the name of the uploaded file.
5. We append the original name of the file using `file.originalname` to maintain some context about the uploaded file. You can modify this function to generate filenames based on your specific needs.
6. After setting up the storage configuration, you create an instance of Multer by calling `multer({ storage })`, passing in the **storage** configuration object. This creates the Multer middleware that you can use in your Express application to handle file uploads.

Example1

Write an express js script that accepts **single file** to be uploaded using the multer middleware and saves the file to the specific directory called "lju".

form.html [both files in same folder]

```
<form action="/uploadfile" method="post" enctype="multipart/form-data">
  <input type="file" name="mypic" accept=".jpg,.jpeg,.png"/>
  <input type="submit" value="Upload"/>
</form>
```

m1.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
const app = express();
app.use(express.static("./", {index: 'form.html'}));

// SET STORAGE
var store = multer.diskStorage({
  destination:"lju",
  filename: function (req, file, cb) {
    cb(null, file.originalname)
  }
})
var upload = multer({ storage: store })
app.post('/uploadfile', upload.single('mypic'), (req, res) => {
  const file = req.file
  if (file) {
    res.send("<h1>File <span style='color:red>'"+ file.originalname + "'</span> has been
    uploaded in <span style='color:red>' + file.destination + " </span>folder")
  }
})
app.listen(6788);
```

Example 2

Write an express js script that accepts **multiple files max number 5** to be uploaded using the multer middleware and saves the files to the specific directory called **“multiple”**.

multiple.html [both files in same folder]

```
<html>
  <form action="/uploadfile" method="post" enctype="multipart/form-data">
    <input type="file" name="mypic" accept=".doc,.docx" multiple>
    <input type="submit" value="Upload"/>
  </form>
</html>
```

multiple.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
const app = express();
```

```

app.use(express.static(__dirname,{index: 'multiple.html'}));

// SET STORAGE
var storage = multer.diskStorage({
  destination:"multiple",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-"+Date.now()+".docx")
  }
// fieldname will change with file name "myfile-the number of milliseconds since January 1, 1970"

});
var upload = multer({ storage })
app.post('/uploadfile', upload.array('mypic',5), (req,res) => {
  const files = req.files
  if (files) {
    res.set("content-type","text/html")
    for(i of files){
      res.write("<h2>File <span style='color:red'>" + i.originalname + "</span> has been
uploaded </h2>")
    }
    res.send()
  }
})
app.listen(6788);

```

Example -3 [both files in same folder]

Write express js script to upload file with **size limit of 10 MB** to a specific directory named "Data" on the server. And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

mult1.html

```

<html>
  <form action="/uploadfile" method="post" enctype="multipart/form-data">
    <input type="file" name="lju" accept=".pdf"/>
    <input type="submit" value="Upload"/>
  </form>
</html>

```

mult1.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
const app = express();

app.use(express.static(__dirname,{index:'mult1.html'}))

// SET STORAGE
var storage = multer.diskStorage({
  destination:"Data",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-"+ "File.pdf")
  }
})
var upload = multer({ storage, limits:{fileSize:10*1024*1024} })
app.post('/uploadfile', upload.single('lju'), (req,res) => {
  const file = req.file
  res.send("<h1 style='color:red'>" + file.originalname + " has been Uploaded</h1>")
})
app.listen(6788);
```

Example - 4

Write an express js script to configure the **multer** middleware. Perform following tasks.

- 1) Create a **html** file named "file.html". This file contains heading(h3) "Upload your CV" in red color. And, a form with input type file(to browse and select file) and submit(to upload the file).
- 2) Create a js file named "file.js" and link this js and html file to browse html file on "/home" page.
- 3) After uploading a file display message on "/upload" page "(file original name) has been uploaded".
- 4) Save uploaded files to specific directory named "upload". And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

file.html [both files in same folder]

```
<body>
  <h3 style="color:red">Upload your CV</h3>
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="lju">
    <input type="submit" value="Submit">
  </form>
</body>
```

file.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
const app = express();
app.use(express.static(__dirname,{index: "file.html"}));

// SET STORAGE
var storage = multer.diskStorage({
  destination:"upload",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-file.pdf")
  }
})
var upload = multer({ storage: storage })
app.post('/upload', upload.single('lju'), (req, res) => {
  const file = req.file
  console.log(file)
  if (file) {
    res.send("<h1>File <span style='color:red>'"+ file.originalname + "'</span> has been
uploaded in <span style='color:red>' + file.destination + " </span>folder")
  }
})
app.listen(6788);
```

In browser

Upload your CV	
Choose File	Chapter 1 .pdf
Submit	
File 'Chapter 1 .pdf' has been uploaded in upload folder	

In console**Output of console.log(file)**

```
{
  fieldname: 'lju',
  originalname: 'Chapter 1 .pdf',
  encoding: '7bit',
  mimetype: 'application/pdf',
  destination: 'upload',
  filename: 'lju-file.pdf',
  path: 'upload\\lju-file.pdf',
  size: 1200577
}
```

Mail send- nodemailer

Nodemailer is a module for Node.js applications to allow easy email sending.

Install nodemailer using the following command:

```
npm install nodemailer
```

Step-1 : require nodemailer

```
const nm = require("nodemailer")
```

Step- -2 : Create first object using .createTransport()

```
var trans = nm.createTransport (options[, defaults] )
```

- This line declares a variable **trans** and assigns it the result of a function call to **createTransport()** method of the **nodemailer** object.
- This indicates that there is likely an imported or required module named **nm** which provides this functionality.
- The purpose of this line is to create an email transport object for sending emails.

Where,

- **options** – is an object that defines connection data (see below for details)
- **defaults** – is an object that is going to be merged into every message object. This allows you to specify shared options, for example to set the same from address for every message.

General options

- **service: "gmail"**: This specifies the email service provider being used, which in this case is Gmail.
- **host: "smtp.gmail.com"**: This specifies the SMTP (Simple Mail Transfer Protocol) server host for Gmail, which is **smtp.gmail.com**.
- **port: 587**: This specifies the port number through which the connection to the SMTP server will be made. Port 587 is commonly used for secure SMTP (SMTPS).
- **secure: true**: This indicates that the connection to the SMTP server should be secure. This typically means using TLS (Transport Layer Security) to encrypt the communication.
- **auth: { ... }**: This object contains authentication credentials required to log in to the SMTP server and send emails.
 - **user: "abc @gmail.com"**: This specifies the username or email address used for authentication.
 - **pass: "16 digit code"**: This specifies the App password for the email account being used for authentication. This App password is provided in plain text, which is not recommended for security reasons. It's safer to use environment variables or other secure methods to store sensitive information like passwords.

Step-3 Create second object and store in variable

```
Var mailoption = ({mailOptions })
```

Parameters

mailOptions: An object containing the details of the email to be sent. The mailOptions object can include the following properties:

- **from:** The sender's email address. This can be a simple email address or an address with a name. (from: 'your-email@gmail.com' // or 'Your Name <your-email@gmail.com>')
- **to:** The recipient's email address. This can be a single email address or a comma-separated list of addresses. (to: 'recipient1@example.com, recipient2@example.com')
- **cc: (Optional)** Carbon copy recipients. This can be a single email address or a comma-separated list of addresses. (cc: 'cc-recipient@example.com')
- **bcc: (Optional)** Blind carbon copy recipients. This can be a single email address or a comma-separated list of addresses. (bcc: 'bcc-recipient@example.com')
- **subject:** The subject line of the email.(subject: 'Subject of your email')
- **text:** The plain text body of the email.(text: 'Hello, this is the plain text body of the email')
- **html: (Optional)** The HTML body of the email.(html: '<p>Hello, this is the HTML body of the email</p>')
- **attachments** - An array of attachment objects. Attachments can be used for embedding images as well.

Step -4 use method .sendMail(mailOptions, callback)

```
trans.sendMail(mailoption,(err,info)=>{....})
```

The .sendMail() method in nodemailer is used to send an email using the specified transport configuration. Here's a detailed explanation of the method and its parameters.

Callback Function: A function to handle the response once the email has been sent or an error has occurred. It takes two arguments:

- **error:** An error object if an error occurred, or null if no error occurred.
- **info:** An object containing information about the sent message, such as the message ID and response.

Example -1 :**Simple example to send mail using nodemailer.**

To start sending emails with Nodemailer, all you need to is:

- Create a transporter object
- Configure the mailoptions object
- Deliver a message with sendMail()

```
var nm = require("nodemailer");
var trans = nm.createTransport({
  host : "smtp.gmail.com",
  port:465,
  auth:{
    user : "sender@gmail.com",
    pass : "App Password "
  }
});
var mailoption = {
  from:"sender@gmail.com",
  to:"receiver1@gmail.com,receiver2@gmail.com ",
  subject : "Hello",
  text : 'Test mail',
  html:'Testing node mailer, <h1>Effect of h1</h1>'
//once html used then text is neglected in most of the case
};
trans.sendMail(mailoption,(err,info)=>{
  if(err){
    console.error(err);
  }
  console.log(info);
});
```

Output: Run node mailer.js in terminal and observe

Example-2

Perform the following tasks as asked:

1) Create a HTML file for response form and this file should be loaded on home('/') page.

•Fields are : Name, Email and Submit button.

2) Once Response is submitted, message “Thank you for your response.” Will be displayed on page ‘/response’ and also send mail to the entered email id with the submitted response.

index.html

```
<form action="/response" method="POST">
  Name: <input type="text" name="name" required><br><br>
  Email: <input type="email" name="email" required><br><br>
  <input type="submit" value="Submit">
</form>
```

app.js

```
const express = require('express');
var nm = require("nodemailer");
const app = express();

// middleware to read form data
app.use(express.urlencoded({ extended: true }));

// serve static files (HTML)
app.use(express.static(__dirname)); // serves index.html automatically

// POST route
app.post('/response', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;

  // transporter
  var trans = nm.createTransport({
    host: "smtp.gmail.com",
    port: 465,
    secure: true,
    auth: {
      user: "sender@gmail.com",
      pass: "your_app_password"
    }
  });

  // mail options
  var mailoption = {
    from: "sender@gmail.com",
    to: email,
    subject: "Response Received",
    text: `Hello ${name}, your response has been received.`,
    html: `Hello ${name},<br><br>
      <h3 style="color:green;">Thank you for your response.</h3>`
  };

  // send mail
  trans.sendMail(mailoption, (err, info) => {
    if (err) {
      console.error(err);
      return res.send("Error sending email");
    }
    console.log(info);
    res.send("<h2>Thank you for your response.</h2>");
  });
});

app.listen(3000, () => {console.log("Server running on port 3000");});
```

Example -2

Perform the following tasks as asked.

- Create a HTML file for feedback form and this file should be loaded on home('/') page.
- Fields are :
name, email, dropdown for the feedback ,text area for the additional comments, and submit button.
- Once feedback submitted, message “Thank you for your feedback.” Will be displayed on page ‘/feedback’ and also send mail to the entered email id with the submitted feedback data. (Data can be submitted using get/post method)

Src/mail_form.js

```
const expr = require("express");
const app = expr();
const nm = require("nodemailer");

// static folder
app.use(expr.static("../public", { index: "mail.html" }));

app.get("/feedback", (req, res) => {
  // store query values
  const { mail, uname, feedback, comments } = req.query;

  // transporter
  const trans = nm.createTransport({
    host: "smtp.gmail.com",
    port: 465,
    secure: true,
    auth: {
      user: "sender@gmail.com",
      pass: "2stepverificationkey"
    }
  });

  // mail options using template literal
  const mailoption = {
    from: "noreply@gmail.com <noreply>",
    to: mail,
    bcc: "bcc@gmail.com",
    subject: "Feedback response",
    html: `<h1>Hello ${uname}!</h1>
    <h2> Thank you for submitting the feedback.<br>
    Your feedback:
    <span style="color:red">${feedback}</span>`
  };
});
```

```

    </h2>
    <h3>Additional comments: ${comments}</h3> `
  };

  // send mail
  trans.sendMail(mailoption, (err, info) => {
    if (err) return console.error(err);
    console.log(info);
  });
  res.send("Thank you for your feedback.");
});
app.listen(5001);

```

/public/mail.html

```

<html>
  <head><title>Feedback Form</title></head>
  <body>
    <form action="/feedback" method="get">
      Name: <input type="text" name="uname"/>
      Email: <input type="text" name="mail"/>
      Feedback: <select name="feedback">
        <option value="bad">Bad</option>
        <option value="good">Good</option>
        <option value="verygood">Very Good</option>
        <option value="excellent">Excellent</option>
      </select>
      Comments: <textarea name="comments" rows="10" cols="15"
placeholder="Comments"></textarea>
      <input type="submit" value="Submit"/>
    </form>
  </body>
</html>

```

View – Engine [EJS]

A template engine like EJS allows you to use static template files in your application. At runtime, it replaces variables in the template with actual values and generates an HTML page that is sent to the client. This makes it easier to create dynamic web pages.

Some popular template engines that work with Express are Pug, Mustache, and EJS.

Why EJS?

EJS (Embedded JavaScript) is a simple and widely used templating engine that lets you embed JavaScript directly into HTML using special tags.

It supports features like:

- Variable interpolation
- Control flow (if/else, loops)
- Partials (includes)
- Reusable templates

Unlike Pug, EJS syntax is very similar to standard HTML, which makes it easier for beginners to understand and use.

Installation

```
npm install ejs
```

Setup in Express

After installing EJS, set it as the view engine in your Express app:

```
app.set('view engine', 'ejs');
```

Key Features of EJS

- Uses plain HTML with embedded JavaScript
- Easy to learn and use
- Supports dynamic data rendering
- Allows reuse of code using includes
- Works seamlessly with Express

EJS lets you write HTML + JavaScript together and render dynamic pages using Express.

For folder structure with “views” folder:

Structure

```
project/
├── app.js
├── views/
│   └── first.ejs
```

Note: In Express.js, the views folder is just the default location where template files (like .ejs) are stored.

When you write: `res.render("first");`

Express automatically looks for: `/views/first.ejs`

app.js

```
const express = require('express');
const app = express();
// set EJS
app.set('view engine', 'ejs');
app.get('/', (req, res) => {
  res.render('first');
});
app.listen(3000);
```

For folder structure without “views” folder:

If you **don't want to use a views folder**, you just need to tell Express where your EJS files are located.

Folder structure

```
project/
├── app.js
└── first.ejs (no views folder needed)
```

You can change the views directory like this in app.js

```
const express = require('express');
const path = require('path');
const app = express();
app.set('view engine', 'ejs');
// set custom folder (same directory or any folder you want)
app.set('views', path.join(__dirname));
app.get('/', (req, res) => {
  res.render('first');
});
app.listen(3000);
```

EJS TAGS

1. `<% %>` → Scriptlet Tag (Logic Only)

Purpose:

- Used for **control flow (if, loop, variables)**
- Does NOT print anything

Example: `first.ejs`

```
<% let age = 30; %>
<% if (age >= 18) { %>
  <p>Adult</p>
<% } else { %>
  <p>Minor</p>
<% } %>
```

Output:

Adult

2. `<%= %>` → Output (HTML Escaped)

Purpose:

- Displays data safely (prevents HTML injection)

Example: add below code in `first.ejs`

```
<% let name = "<b>ABC</b>"; %>
<p><%= name %></p>
```

Output:

`<b>ABC</b>` (shown as text, NOT bold)

Tags are escaped → safe

3. `<%- %>` → Output (Unescaped HTML)

Purpose:

- Displays raw HTML (renders tags)

Example: add below code in `first.ejs`

```
<% let name = "<b>ABC</b>"; %>
<p><%- name %></p>
```

Output:

ABC (in bold)

Note:

`<%= %>` → shows **text only (safe)** : HTML tags are NOT applied

`<%- %>` → shows **real HTML** : HTML tags ARE applied

4. `<%# %>` → Comment Tag

Purpose:

- Add comments that are **not visible in browser**

Example:

```
<%# This is a comment %>
<p>Hello</p>
```

Output:

Hello

5. `<%% %>` → Literal Tag (escape EJS syntax)

Purpose:

- Used when you want to show EJS syntax as text

Example:

```
<p><%% %> This is EJS %></p>
```

Output:

`<% This is EJS %>`

EJS TAGS

Tag	Description	Example	Output
<code><% %></code>	Scriptlet (logic only, no output)	<pre><% if(x>10){ %> OK <% } %></pre>	OK
<code><%= %></code>	Output (HTML escaped)	<code><%= "Hi" %></code>	<code>Hi</code>
<code><%- %></code>	Output (unescaped HTML)	<code><%- "Hi" %></code>	Hi
<code><%# %></code>	Comment (not executed)	<code><%# comment %></code>	Not visible in browser
<code><%% %></code>	Print literal <code><%</code>	<code><%% Hello %></code>	<code><% Hello %></code>

Example:

Write an Express application using EJS that renders a page displaying “**Hello ABC**” by passing data from the server to the view.

app.js

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  res.render('first', { name: "ABC" });
});
app.listen(3000);
```

views/first.ejs

```
<h1>Hello <%= name %></h1>
```

FORM HANDLING WITH EJS + EXPRESS

It combines server-side rendering with request processing to build dynamic, user-interactive web applications. Using Express.js, form data submitted via GET or POST is easily captured through req.query or req.body, while EJS allows this data to be rendered back into HTML views. This approach is needed to process user inputs such as login, feedback, or registration forms. It offers advantages like clean separation of logic and presentation, easy data binding, reusable templates, and fast development with minimal complexity, making it ideal for building simple to moderately dynamic web applications.

Example -1

Develop a web application using Express and EJS that allows a user to enter a student’s name and marks through a form. After submission, the application should display the entered details and determine the result: if the marks are 40 or above, show “Pass” in green color; otherwise, show “Fail” in red color.

Create Form (form.ejs)

```
<h2>Student Form</h2>
<form action="/submit" method="post">
  Name: <input type="text" name="name"><br><br>
  Marks: <input type="number" name="marks"><br><br>
  <input type="submit" value="Submit">
</form>
```

Server Code(my.js)

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');

// to read form data
app.use(express.urlencoded({ extended: true }));

// show form
app.get('/', (req, res) => {
  res.render('form');
});

// handle form

app.post('/submit', (req, res) => {
  const name = req.body.name;
  const marks = req.body.marks;
  res.render('result', { name, marks });
});

app.listen(3000);
```

Result Page (result.ejs)

```
<h2>Name: <%= name %></h2>
<h2>Marks: <%= marks %></h2>
<% if (marks >= 40) { %>
  <p style="color:green">Pass</p>
<% } else { %>
  <p style="color:red">Fail</p>
<% } %>
```

Example -2

Write an Express.js application to perform the following tasks:

1) Create an EJS file named contact.ejs that displays a heading "Contact Form" and contains a form with:

- Text input for Name
- Email input for Email
- Submit button

2) Render this EJS file on the /contact route.

3) Handle form submission using the POST method on the /submit route.

4) After form submission, display a message on the browser in the format:

"Thank you (Name), we have received your email (Email)." Through ejs file.

Folder Structure

```
project/
├── app.js
├── views/
│   ├── contact.ejs
│   └── result.ejs
```

views/contact.ejs

```
<h2>Contact Form</h2>
<form action="/submit" method="POST">
  Name: <input type="text" name="name" required><br><br>
  Email: <input type="email" name="email" required><br><br>
  <input type="submit" value="Submit">
</form>
```

app.js (Express Server)

```
const express = require('express');
const app = express();

// set EJS
app.set('view engine', 'ejs');
// middleware to read form data
app.use(express.urlencoded({ extended: true }));
// render contact form
app.get('/contact', (req, res) => {
  res.render('contact');
});
// handle form submission
app.post('/submit', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;

  // render result page with data
  res.render('result', { name, email });
});

app.listen(3000, () => {console.log("Server running on port 3000");});
```

views/result.ejs

```
<h2>
  Thank you <%= name %>, we have received your email <%= email %>.
</h2>
```

Example -3

Develop a web application using Express and EJS to accept marks of four tests (T1, T2, T3, T4), each out of 25, through a form using the POST method. After submission, display all entered marks in a tabular format along with the total marks, and determine the result: if the total is **35 or more**, display **“Pass” in green color**; otherwise, display **“Fail” in red color**. (result also by ejs file)

```
project/
├── app.js
├── views/
│   ├── form.ejs
│   └── result.ejs
```

views/form.ejs

```
<h2>Enter Marks (Each out of 25)</h2>

<form action="/submit" method="POST">
  T1: <input type="number" name="t1" required><br><br>
  T2: <input type="number" name="t2" required><br><br>
  T3: <input type="number" name="t3" required><br><br>
  T4: <input type="number" name="t4" required><br><br>
  <input type="submit" value="Submit">
</form>
```

views/result.ejs

```
<h2>Marks Details</h2>
<table border="1" cellpadding="10">
  <tr>
    <th>T1</th>
    <th>T2</th>
    <th>T3</th>
    <th>T4</th>
  </tr>

  <tr>
    <td><%= t1 %></td>
    <td><%= t2 %></td>
    <td><%= t3 %></td>
    <td><%= t4 %></td>
  </tr>
</table>
<br>
<h2>Total: <%= total %></h2>
<% if (total >= 35) { %>
  <h3 style="color:green;">Pass</h3>
<% } else { %>
  <h3 style="color:red;">Fail</h3>
<% } %>
```

app.js

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');

// middleware for POST data
app.use(express.urlencoded({ extended: true }));

// show form
app.get('/', (req, res) => {
  res.render('form');
});

// handle form
app.post('/submit', (req, res) => {
  const t1 = parseInt(req.body.t1);
  const t2 = parseInt(req.body.t2);
  const t3 = parseInt(req.body.t3);
  const t4 = parseInt(req.body.t4);
  const total = t1 + t2 + t3 + t4;
  res.render('result', { t1, t2, t3, t4, total });
});
app.listen(3000);
```

Example - 4

Create a basic web application using Express.js, and EJS to display user information on a webpage.

The application should:

- Configure EJS as the view engine.
- Create a route /users in the server.
- Inside this route, define a list of users (each user should have name, email, and age).
- Pass this user data from the server to an EJS file.
- Use EJS to loop through the data and display it in an HTML table.

my.js

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');

// sample JSON data
app.get('/users', (req, res) => {
  const users = [
    { name: "Rahul", email: "rahul@gmail.com", age: 22 },
    { name: "Priya", email: "priya@gmail.com", age: 25 },
    { name: "Amit", email: "amit@gmail.com", age: 30 }
  ];
```

```
res.render('my', { users });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

my.ejs

```
<h2>User List</h2>

<table border="1" cellpadding="10">
  <tr>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
  </tr>

  <% for(let u of users) { %>
    <tr>
      <td><%= u.name %></td>
      <td><%= u.email %></td>
      <td><%= u.age %></td>
    </tr>
  <% } %>
</table>
```

User List

Name	Email	Age
Rahul	rahul@gmail.com	22
Priya	priya@gmail.com	25
Amit	amit@gmail.com	30